

UNIVERSITATEA "ALEXANDRU-IOAN CUZA" DIN IAȘI

**FACULTATEA DE INFORMATICĂ**



LUCRARE DE DISERTAȚIE

**Streamlining Blockchain security audits  
through desktop applications**

propusă de

**Marius Enache-Stratulat**

**Sesiunea: Iulie, 2025**

Coordonator științific

**Conf. Dr. Andrei Arusoaie**

UNIVERSITATEA "ALEXANDRU-IOAN CUZA" DIN IAȘI

**FACULTATEA DE INFORMATICĂ**

# **Streamlining Blockchain security audits through desktop applications**

**Marius Enache-Stratulat**

**Sesiunea: Iulie, 2025**

Coordonator științific

**Conf. Dr. Andrei Arusoaie**

Avizat,  
Îndrumător lucrare de disertație,  
Conf. Dr. Andrei Arusoaie.

Data: ..... Semnătura: .....

### **Declarație privind originalitatea conținutului lucrării de licență**

Subsemnatul **Enache-Stratulat Marius** domiciliat în **România, jud. Bacău, mun. Bacău, strada Milcov, nr. 65, bl. 65, ap. 4**, născut la data de **31 mai 2001**, identificat prin CNP **5010531046224**, absolvent al Facultății de informatică, **Facultatea de informatică** specializarea **Securitatea Informatiei**, promoția 2025, declar pe propria răspundere cunoscând consecințele falsului în declarații în sensul art. 326 din Noul Cod Penal și dispozițiile Legii Educației Naționale nr. 1/2011 art. 143 al. 4 și 5 referitoare la plagiat, că lucrarea de licență cu titlul **Streamlining Blockchain security audits through desktop applications** elaborată sub îndrumarea domnului **Conf. Dr. Andrei Arusoaie**, pe care urmează să o susțin în fața comisiei este originală, îmi aparține și îmi asum conținutul său în întregime.

De asemenea, declar că sunt de acord ca lucrarea mea de licență să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului ei într-o bază de date în acest scop.

Am luat la cunoștință despre faptul că este interzisă comercializarea de lucrări științifice în vederea facilitării falsificării de către cumpărător a calității de autor al unei lucrări de licență, de diplomă sau de disertație și în acest sens, declar pe proprie răspundere că lucrarea de față nu a fost copiată ci reprezintă rodul cercetării pe care am întreprins-o.

Data: .....

Semnătura: .....

## **Declarație de consimțământ**

Prin prezenta declar că sunt de acord ca lucrarea de licență cu titlul **Streamlining Blockchain security audits through desktop applications**, codul sursă al programelor și celelalte conținuturi (grafice, multimedia, date de test, etc.) care însoțesc această lucrare să fie utilizate în cadrul Facultății de informatică.

De asemenea, sunt de acord ca Facultatea de informatică de la Universitatea "Alexandru-Ioan Cuza" din Iași, să utilizeze, modifice, reproducă și să distribuie în scopuri necomerciale programele-calculator, format executabil și sursă, realizate de mine în cadrul prezentei lucrări de licență.

Absolvent **Marius Enache-Stratulat**

Data: .....

Semnătura: .....

# Contents

|                                  |    |
|----------------------------------|----|
| Motivation                       | 2  |
| Introduction                     | 1  |
| 1 Threats of Blockchain          | 3  |
| 1.1 Slither . . . . .            | 8  |
| 2 Slither’s performance analysis | 11 |
| 3 SmartScan                      | 15 |
| Concluzii                        | 22 |

# Motivation

The blockchain technology [15] is relatively new if we consider the release of the Bitcoin cryptocurrency blockchain network, which took place in 2009. However, its foundation was formed decades ago, in 1989, when Leslie Lamport developed the Paxos protocol [8], a distributed consensus mechanism that became the base of many decentralized systems' implementation.

While it may be primarily associated to cryptocurrencies, blockchain has grown broader in terms of its applications. Its infrastructure's main qualities are the decentralized trust, transparency and immutability, which make it perfect in domains such as electronic voting, digital identity or finance. However, one of its advantages is also a critical shortcoming, because, once deployed, a smart contract cannot be modified, leaving the door open to attackers if there is any vulnerability to exploit, which is almost impossible to avoid completely.

This thesis' motivation is to address the most vulnerable sector of the blockchain community, the beginner developers and students. We propose a desktop application that integrates Slither, a fast and powerful security analysis tool, and GitHub's REST API to automatically scan smart contracts and provide an overall security report and a visual guide towards the vulnerabilities. This application should act as a training wheel for the beginner developers, but it can also be paired with a thorough manual analysis for the more advanced ones.

We hope that this kind of tool will improve the security and efficiency of smart contracts for the long term, which will prevent unnecessary electricity consumption and funds losses. While no security analysis tool is perfect and can miss certain vulnerabilities, it can prevent obvious and high-impact attacks from taking place and alleviate the developers' pressure to deliver secure smart contracts.

## **Abstract**

The Blockchain, despite its popularity and presence on the internet and in the financial sector, is a relatively new technology and this fact is proved by the large amount of attacks that took place on such networks over the years. This situation raises the need for secure and robust networks that can keep the clients' financial assets safe. Fortunately, there are various solutions to this category of problems, like Slither or Manticore, which take different approaches towards finding vulnerabilities. For the moment, most of these tools provide an at least decent detection for a high variety of attacks, but they need to get used more in order to ensure a higher standard of security when it comes to smart contracts. A good way to achieve this is to integrate the scanning tools into user-friendly applications that shorten the process of analyzing and fixing vulnerabilities.

# Introduction

The 21st century, so far, featured many advancements in the technology field. In the first decade, the internet had its peak spread among the high majority of people, the second one had seen a large growth of the smartphone's adoption across the globe and now, in the third decade, the artificial intelligence is the most used and discussed breakthrough. The blockchain, while more niche than the other technologies, also gained very much traction in the last 10 years, especially since 2020, when many people tried to take advantage of the sudden rise of Bitcoin, in both popularity and value, through mining or trading.

The blockchain is a decentralized, distributed and immutable ledger in which every user can add new blocks without the need of a central authority. Its foundation as a concept can be traced back to Leslie Lamport's Paxos protocol [8], released in 1998, which describes a system designed to achieve consensus among a network of computers, which is needed for agreeing on each iteration of the ledger, as it cannot be modified after its submission.

These systems, starting with the release of Ethereum, allowed the development of smart contracts [7], which are self-executing applications stored on the blockchain. The instructions of a smart contract are triggered when certain conditions are met and can be called without the need of a server. This property, along with its deterministic, transparent and immutable nature, gives the user the most power and responsibility possible.

This technology has emerged only about 16 years ago, which can be seen because of the multitude of attacks that affected the networks that are based on it. This is why there is a need for secure and resilient networks that can prevent the loss of clients' assets. Thankfully, there are many different tools that alleviate this concern, such as Slither [6] and Manticore [11], which find vulnerabilities in smart contracts in different ways. The aim of this thesis is to show Slither's strengths and introduce SmartScan,



an open-source desktop application that integrates it in a more seamless process of detecting and fixing security errors. This application is especially meant for students and aspiring smart contract developers.

# Chapter 1

## Threats of Blockchain

The Blockchain [13] is a technology used in various applications, such as cryptocurrencies, Internet of Things, electronic voting and many others. Its transparent, fully distributed, peer to peer and append-only nature is its most notable strength. The transactions, which may have a different meaning depending on the application they are used for, are publicly visible and cannot be modified after they are published. This means the users may verify any transaction with no need for a centralized authority.

Bitcoin, the most popular cryptocurrency at this moment, takes advantage of these characteristics, creating a decentralized financial system. Unfortunately, there are also significant shortcomings in terms of security for this kind of networks. One of the most notorious instances where these vulnerabilities had been exploited is known as “The DAO” [13], where in 2016 an unknown attacker managed to drain \$50 million USD through a *Reentrancy Attack*.

This thesis will first present the variety of attacks that blockchain networks are vulnerable to, their manner of work and potential effects to its victims. The Quadriga Initiative [23] - a community-based platform - hosts a list of most of the attacks and frauds that involve cryptocurrency exchanges. Among the most well-known attacks, the database of case studies includes instances of Reentrancy Attacks, Replay Attacks and Short Address Attacks.

*Reentrancy Attacks* [5] can take place whenever the developers of the targeted network do not update the balances before sending the funds. In this case, an attacker could recursively call the `withdraw()` method and drain all the available funds.

*Replay Attacks* [14] are specific to the case where a cryptocurrency is forked into two separate currencies. In such a scenario, it is possible to sniff a regular transaction

from any of the chains and “replay” it on the other. Processing both transaction causes the user to lose the same amount of assets twice (for example, paying two ether instead of one for the same product or service). This attack was easily possible in the Ethereum blockchain before the implementation of chainID in transactions.

*Short Address Attacks* [13] represent the exploit of a bug in the EVM – Ethereum Virtual Machine – used to obtain extra tokens on purchases. This category of attacks requires creating an Ethereum wallet with its address ending with a 0 byte. The attacker makes a purchase on the address by removing the last digit and causes the contract to try to append the missing byte to the incomplete address, but it ends up appending it to the paid amount. In this case, the contract pays 256 times more tokens than intended.

In terms of prevention and detection of such attacks, there are numerous tools that serve this purpose. As Kaixuan Li et al. describe in their article [10], a significant category of such tools detect vulnerabilities in the code through either static analysis or symbolic execution.

| Technology         | Tool       | Analysis Level | Stars |
|--------------------|------------|----------------|-------|
| Static Analysis    | Securify2  |                | 529   |
|                    | Slither    | Source Code    | 4 500 |
|                    | SmartCheck |                | 315   |
| Symbolic Execution | Manticore  |                | 3 500 |
|                    | Osiris     | Bytecode       | 50    |
|                    | Oyente     |                | 1 300 |

Table 1.1: List of the main static analysis and symbolic execution tools, adapted from Kaixuan Li et al. [10]

According to table 1.1, the static analysis tools, primarily represented by Slither, enjoy a higher popularity than the symbolic execution tools, represented by Manticore. This fact is shown by the number of stars on GitHub, which indicate how popular a tool is. These tools share the principle of scanning the code without executing it, but do so in different ways.

Generally, static analysis tools [4] are automatic methods that determine the run-time properties of a program without running it. In Slither’s [6] case, the analysis’ purpose is to detect security vulnerabilities within the smart contract and to point out potential optimizations, which result in a lower gas consumption, an essential aspect

in this field. As such, this category of security tools should be considered the opposite of Dynamic Analysis tools, which rely on running the target program and can also monitor its performance among other aspects.

On the other hand, symbolic execution tools [1] take a similar approach of analyzing the given program without running the code, based solely on its bytecode. Essentially, Manticore [11] uses the bytecode of a smart contract to map out all the execution paths of the program. Afterwards, all the paths get explored and the tool determines what kind of inputs determine each possible outcome. Two of the main components of Manticore are the Core Engine and the Event System. The Core Engine is responsible with operating and managing the state of a program at a certain point in the execution. The Execution Module has the purpose to emulate a system that runs the target program, with a CPU, system memory and an operating system like Linux.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract FoobarContract {
    function foobar(int a, int b) public pure returns (int) {
        int x = 1;
        int y = 0;

        if (a != 0) {
            y = 3 + x;
            if (b == 0) {
                x = 2 * (a + b);
            }
        }

        require(x - y != 0, "Assertion failed: x - y == 0");

        return x;
    }
}
```

Figure 1.1: Example of code to analyze. Adapted from Roberto B et al. [1]

The figure above shows a simple example of function that a symbolic execution tool can operate on. A tool like Manticore can trace all the possible paths the execution may go at run-time. The assertion at the end is the possible point of failure for this function, in case  $x$  and  $y$  are equal.

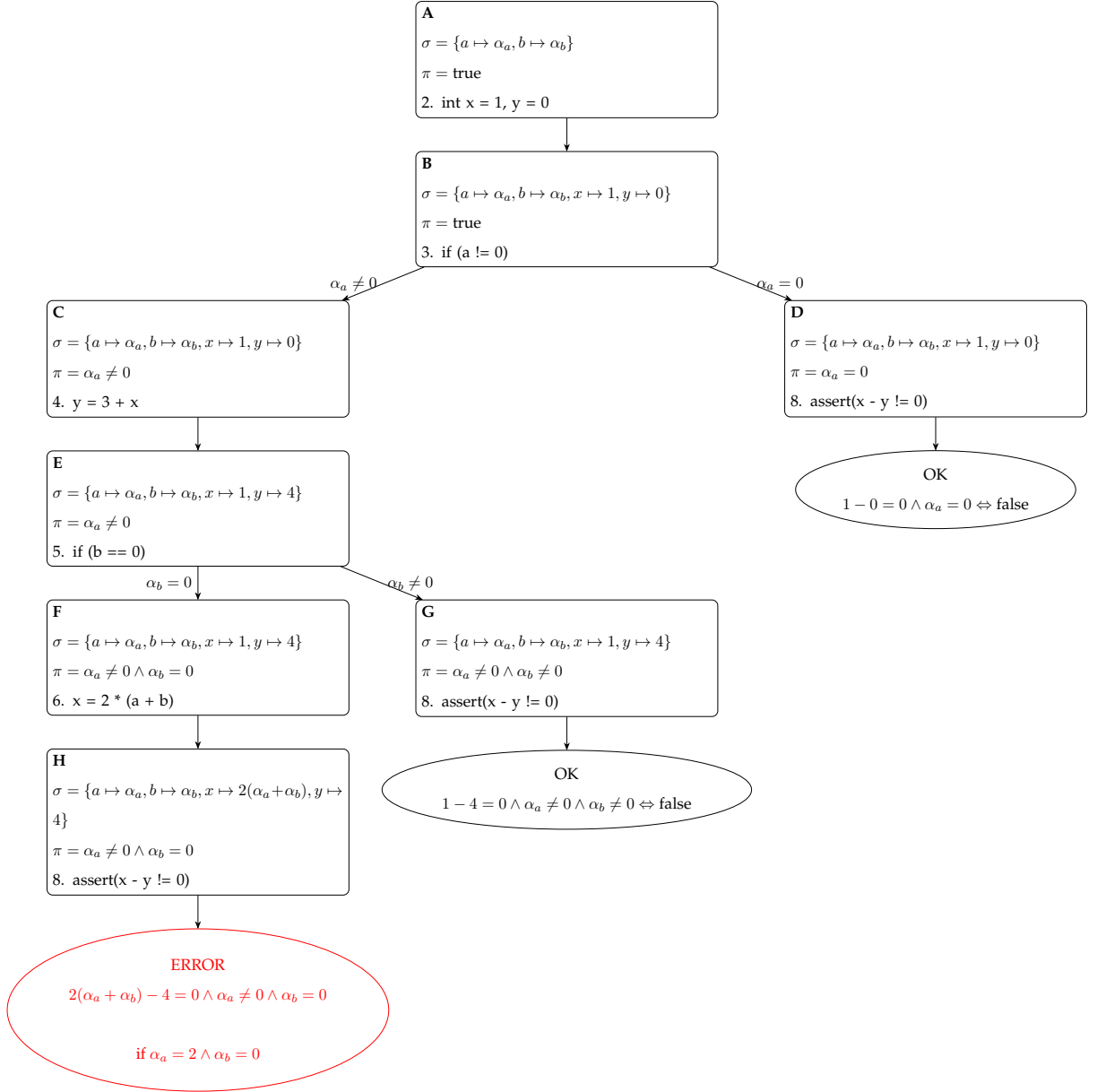


Figure 1.2: Example of symbolic execution tree of function `foobar`. Each state shows the symbolic store  $\sigma$ , the path constraint  $\pi$ , and the current statement. From Roberto B. et al [1]

As shown in this graph,  $\sigma$  contains all the symbols stored in the context of the function,  $\pi$  is a statement that represents the path constraint and below it is the line of code that gets executed at a given time. State *B* is the first one to fork in two hypothetical states, based on the value of the first parameter, as shown by  $\pi$  in the states *C* and *D*. If  $a == 0$ , the function skips to the final assert and returns with no failure. Otherwise, the program would continue with state *E* and the following ones. Finally, a symbolic execution tool can reveal that the function will throw an error in case  $a == 2$  and  $b == 0$ .

| Tool      | # Success    | # Failure |               |         |
|-----------|--------------|-----------|---------------|---------|
|           |              | # Timeout | # Compilation | # Total |
| Slither   | 767 (97.34%) | 2         | 19            | 21      |
| Manticore | 112 (14.21%) | 626       | 50            | 676     |

Table 1.2: Success rate and failures summary of Slither and Manticore. Adapted from Kaixuan Li et al. [10]

Finally, to address Manticore’s performance, as shown above, its exhaustive approach causes it to fail most of the tests due to timeout. Even if we were to ignore the timeout fails, Slither is visibly more consistent, having less compilation failures. The main flaw and reason of Manticore’s high timeout rate may be the fact that it has to process large amounts of possible paths in order to find the ones where a security flaw is exploited. As such, Slither is the more consistent and better performing in comparison to Manticore.

## 1.1 Slither

Considering the performance metrics discussed in the first section, this section will be dedicated to Slither [6], a static analysis tool catered to smart contracts written in Solidity and designed to offer granular information about smart contract code. This tool specializes in vulnerability detection, but also manages to point out potential optimizations that lower the gas consumption of a smart contract. In order to facilitate the usage of this tool, Slither leverages printers to help the user better understand the contract’s functionality and structure. This is done through representations such as the control flow graph or the inheritance graph or summaries of the contract and the issues detected in its code.

In opposition to dynamic analysis [3] tools, which rely on executing the contract’s code to find security weaknesses, static analysis tools detect vulnerabilities without running the smart contract. Such approaches include parsing and decompiling the EVM bytecode and translating it into semantic facts. These semantic facts are then matched with predefined patterns meant to reveal common issues.

Slither [6] proceeds in a similar manner by taking the Solidity Abstract Syntax Tree generated by the compiler and extracting information such as the control flow graph and transforms the code given into SlithIR, its internal representation language.

After these two initialization stages, the actual code analysis takes place. In this stage, Slither identifies the reads and writes of variables, analyses the data dependency and determines whether the methods are protected from unauthorized use (for instance, only the owner of a contract may create new tokens).

This tool [6] helps auditors detect various vulnerabilities and potential security threats, such as reentrancy attacks or uninitialized storage variables. Its approach of quickly translating and analyzing large codebases makes it significantly faster than symbolic execution engines, as shown Table 1.2, a great feature for fast development workflows.

Furthermore, its set of detectors are able to find obvious and subtle vulnerabilities alike while providing useful educational resources to help developers fix and prevent future attacks. All these factors make Slither a great solution for quick and automated daily audit processes, preventing many critical security vulnerabilities.

Another essential topic in blockchain development, beside security concerns, is optimization, the techniques used to lower the “gas” consumption. Slither includes a set of detectors specialized in finding ways to optimize code [21] from different perspectives. They are able to find issues such as dead code, as in methods or code branches that are never called or executed, unused state variables or variables that should be declared as constants to save gas.

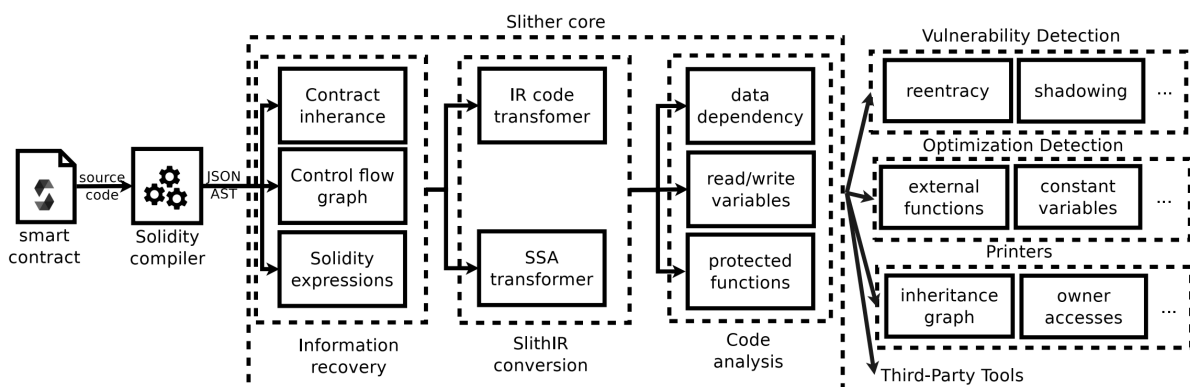


Figure 1.3: Slither overview. From Josselin F. et al. [6]

As to how it works [6], Slither uses a multi-stage procedure to parse and process the codebase of a blockchain project. First, it uses the Solidity compiler to generate the Abstract Syntax Tree of the contract, from which it recovers important information: inheritance graph, control flow graph and the list of expressions. Next, the contracts get translated to SlithIR, the internal representation language. The following step is the ac-



tual code analysis, in which Slither processes data dependencies, read/write variables and protected functions to output detected vulnerabilities and optimization opportunities. Lastly, the tool prints all the found vulnerabilities, showing the section of code that causes it.

As we can see, Slither's approach is to analyze the patterns within the project's code and match them with the ones it deems dangerous or unoptimized. This way, this tool doesn't need to scan all the possible execution paths of the program to find security weaknesses and manages to provide consistent results with little to no failures.

## Chapter 2

### Slither's performance analysis

This chapter will be dedicated to assessing Slither's performance in terms of vulnerability detection. The metrics used to determine how well security scanning tools work are: detection rate (how often a tool can correctly flag a vulnerability), analysis speed and attacks coverage (the variety of attacks that a tool can prevent).

|                     |                         | Slither      | Securify        | SmartCheck      | Solhint         |
|---------------------|-------------------------|--------------|-----------------|-----------------|-----------------|
| Accuracy            | False Positive Rate     | 10.9%        | 25%             | 73.6%           | 91.3%           |
|                     | Flagged contracts       | 112          | 8               | 793             | 81              |
|                     | Detections per contract | 3.17         | 2.12            | 10.22           | 2.16            |
| Performance         | Execution time(s)       | $0.79 \pm 1$ | $41.4 \pm 46.3$ | $10.9 \pm 7.14$ | $0.95 \pm 0.35$ |
|                     | Time out rate           | 0%           | 20.4%           | 4%              | 0%              |
| Robustness          | Failure rate            | 0.1%         | 11.2%           | 10.22%          | 1.2%            |
| Reentrancy examples | DAO                     | Yes          | No              | Yes             | No              |
|                     | Spankchain              | Yes          | No              | No              | No              |

Table 2.1: Performance comparison between Slither, Securify, SmartCheck and Solhint. From Josselin F. et al. [6]

The table above covers the first two performance criteria: the detection rate and analysis speed. The experiment was done by analyzing 1000 contracts focusing only on reentrancy detectors. Slither's results are visibly better than the ones obtained by its counterparts. Firstly, its false positive rate is far lower, at just 10.9%, while Securify falls in the second place with 25%. Secondly, its average execution time is also significantly lower than those of Securify and SmartCheck. In short, Slither is by far faster, more accurate and more consistent than the other three tools. Another important aspect is the low time out rate of for Slither and Solhint, which further supports Slither's

consistency in the detection of reentrancy attacks. Also, while this results do not conclude Slither’s attack coverage, we can see it is the only tool that managed to detect attacks similar to the ones that targeted DAO or Spankchain, which are notorious for the amount of funds stolen and its effect on people’s perception towards blockchain’s security.

| Vulnerabilities | Read  | TP | TN | FP | FN |
|-----------------|-------|----|----|----|----|
| Re-entrancy     | 29/31 | 28 | 0  | 0  | 1  |
| Access Control  | 18/18 | 14 | 0  | 0  | 3  |
| Arithmetic      | 15/15 | 0  | 0  | 5  | 0  |
| Unchecked LLC   | 26/26 | 26 | 0  | 0  | 3  |
| Total           | 88    | 68 | 0  | 5  | 7  |

Table 2.2: Slither’s coverage of security attacks. Adapted from Senan B. [2]

This table shows how Slither can detect four types of vulnerabilities: Reentrancy, Access Control, Arithmetic and Unchecked LLC and covers 4 output categories: True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN). Out of the 90 test cases given, 88 passed the reading phase, 68 had got correctly flagged as vulnerable, five ended up as False Positives and seven as False Negatives. That equates to a rate of 77.27% for True Positive, 5.68% for False Positive and 7.95% for False Negative. Therefore, while in almost 6% of cases the user is falsely lead to believe a section of his code is vulnerable, a case that has no impact on the project’s security, the rate at which Slither misses a vulnerability is almost 8%.

| Vulnerabilities | Slither | SmartCheck |
|-----------------|---------|------------|
| Bad Randomness  | 2.99s   | 8.9s       |
| Access Control  | 2.98s   | 10s        |
| Reentrancy      | 2.48s   | *32.2s     |
| Arithmetic      | 3.09s   | 21.16s     |

Table 2.3: Detection rate of Slither compared to SmartCheck, another static analysis tool. \*SmartCheck registered a significantly lower detection rate for reentrancy attacks. Adapted from [9]

According to the table above, Slither is by far faster than SmartCheck at detecting vulnerabilities across all four types included. Also, both tools properly detected the

vulnerabilities, so the quality of Slither's detection is just as high as SmartCheck's, except for reentrancy attacks, where SmartCheck missed a significant part of the issues. In short, Slither is much faster and more reliable than SmartCheck.

Among the included attacks, there are two new types: bad randomness and arithmetic. Bad randomness [12] attacks may take place when a contract generates random numbers through unsafe methods, such as using the block's timestamp as the seed for essential operations, like asset transfer. If the attacker is able to predict the seed (for example, by initiating a transaction at a certain time), they may get an undeserved advantage over honest users and could obtain more funds, depending on the attacked application.

Arithmetic attacks [24] are specific to smart contracts that use versions of Solidity older than Solidity 0.8. This type of attack is done by overflowing or underflowing a variable. Overflowing a variable is done by incrementing it over its limit (for an uint8 variable, adding one to 255 will return 0). Underflowing is done similarly through decrementation. Fortunately, newer versions of Solidity throw errors in these cases and the older ones have access to the SafeMath library, so these vulnerabilities are easy to solve.

As seen in the previous paragraphs, Slither is a great tool for detecting security vulnerabilities, consistently finding issues almost regardless of their category. This high detection rate is backed up by its high speed of processing smart contracts, which is another strong point of Slither. Furthermore, unlike its competition, this tool is also able to detect optimization opportunities in smart contracts.

Smart contract optimization, while not a major security concern, used to be an important subject for Ethereum-based smart contracts developers. This was the case before The Merge, when Ethereum transitioned from the Proof-of-Work consensus to a Proof-of-Stake one. The Proof-of-Work model relied on validators using large amounts of electricity through mining in order to verify transactions and an inefficient smart contract on top of that would only have worsened the problem. However, the shift towards the Proof-of-Stake consensus meant that Ethereum would dramatically lower its energy consumption by over 99.9% on september 14th 2022 [18].

Now, because the energy consumption is not as threatening, optimizing a smart contract is more of a matter of reducing the risk of intruders using inefficient code as a backdoor for their attacks and ensuring a fast and responsive application for its users. As far as the latest researches go, Slither [6] had been tested for detecting constant

variables that were not declared accordingly. A properly declared constant variable will take no space in the storage of the contract and can be accessed with fewer instructions when needed. However, a non-constant variable that acts as a constant (it doesn't change its value under any circumstance) could unnecessarily increase the code size and the usage cost. Slither was tested on a total of 36.000 contracts and up to 56% of them were found to have variables that acted as constants or variables that ended up not being used at all. These findings prove that Slither is able to detect basic, but useful, code optimization.

Thanks to its optimization detection and coverage of various categories of attacks, Slither is a great tool for security audits. However, it can also be used for educational purposes in blockchain related courses, where the students could run it on their projects and see its security vulnerabilities. The usage of Slither in such early stages of learning smart contract development raises awareness of the most common attacks and helps the student avoid leaving the project vulnerable in the future.

Another subject of great importance when discussing Slither, or any similar tool, are the legal aspects and the ethical considerations of using it. The first and most important matter is the fact that, while the developers of a smart contract may run Slither on their project to find and fix its vulnerabilities, intruders may do the same to find exploitable pieces of code to focus their attack on. Of course, this is mostly possible in the case of an open-source project, unless the developers are quick enough to detect and fix all the security problems before any attacker manages to exploit them. In other words, Slither may be both the guard that points out security issues and the complice that helps the attacker find the right way to exploit the contract.

Similarly, regular users can run Slither on open-source smart contracts to assess whether they are safe to use and may contact the developers in case they find glaring problems. In the meantime, developers should take responsibility for the security of their project and conduct manual analysis of the code on top of using automated tools. In the end, security analysis tools are far from perfect and are subject to throwing false positives or negatives. As such, the developers are the only ones responsible with avoiding security attacks to the best of their abilities.

# Chapter 3

## SmartScan

SmartScan [19] is a desktop application that aims to help begginer developers evaluate their projects from a security standpoint while it can also help regular users evaluate how safe a smart contract is. The main purpose of this project is to be easy to use, but also powerful enough to provide relevant insight and guidance towards fixing the security vulnerabilities. For this purpose, SmartScan uses the GitHub REST API [22] for a quick and intuitive project cloning process and Slither [21] for the analysis of the cloned project.

The choice for the analysis tool was not difficult for multiple reasons, as conveyed by the previous chapter. The first one is in its core design: being a static analysis tool makes it great for scanning projects without executing them, which would have added complexity to the entire setup. Secondly, its balance between high performance and low execution time in terms of scanning for security threats is great for providing fast and reliable vulnerability reports.

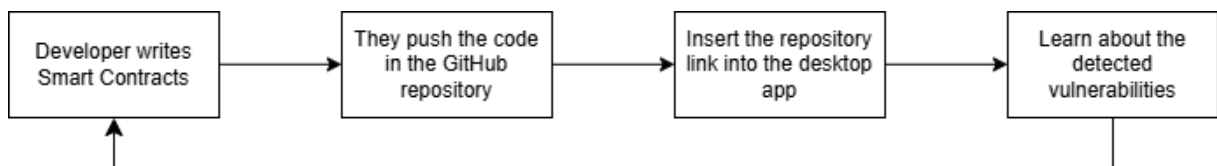


Figure 3.1: Scenario diagram for SmartScan.

As shown in the diagram above, the usual workflow in which SmartScan is involved starts with the developer writing the smart contract code and pushing it in the GitHub repository. Our application intervenes right after, when the developer copies the repository link in it and starts the analysis. Afterwards, the application returns a security report, based on which the developer starts fixing the vulnerabilities and the

cycle may repeat. Whenever the application runs the analysis, it makes sure to fetch the latest version of the project so the developer doesn't need to.

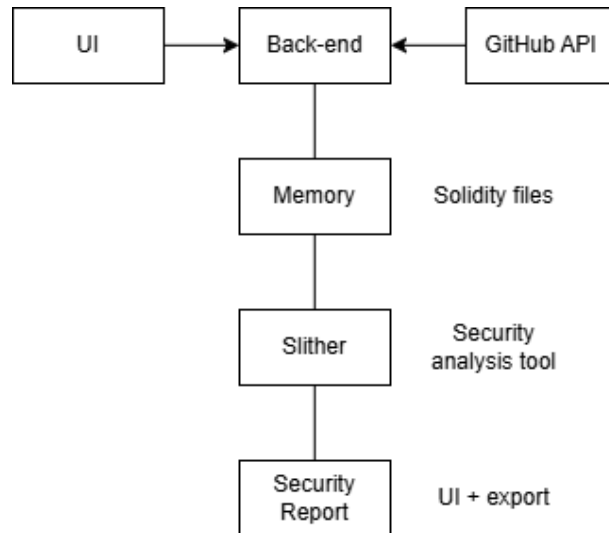


Figure 3.2: Component diagram for SmartScan.

In terms of infrastructure and design choices, SmartScan is developed in Python, using the PySide 6 framework [25] for the interface because it provides a clean and easy to use design with minimal performance limitations. The back-end of the application consists of the aforementioned REST API and Slither, linked together with custom scripts designed to manage the analysis results.



Figure 3.3: SmartScan's User Interface.

The figure above presents the user interface of SmartScan. The first component the user will interact with is the Repository Link text box, in which they will insert the repository link copied from GitHub before pressing the Analyze Button. Afterwards, the REST API will handle the cloning of the project and Slither will analyze it, returning all the found vulnerabilities. The File Tree is used to access the cloned project and open any Solidity file in order to see the code inside. The Code Area will host the code of the currently open file, with a side bar that shows the number of each line and a yellow highlight that shows which lines of code are vulnerable, as detected by Slither. The side bar and the highlight are meant to visually guide the developer towards the problem with ease. After fixing the code, the user may save the file through the dedicated button on the top left corner.

SmartScan, while using external APIs for cloning and analyzing the project, compiles a security report of its own, based on the vulnerabilities found by Slither. The whole report will be saved in a separate text file, so the user can access it whenever they want, and will be shown in the Code Area at the end of the analysis. Also, the stars next to the Save Button will turn red depending on the determined severity of the project. The metric used to measure how vulnerable a project is will be the amount of threats found and their severity. Each star turning red represents a higher vulnerability of the project as a whole. Ideally, the analyzed project should get a rating of zero stars, while a five-star rating is the worst case possible.

| Star rating | Condition                                                              |
|-------------|------------------------------------------------------------------------|
| 0 ★         | No vulnerability found                                                 |
| 1 ★         | $\leq 10$ vulnerabilities, no high severity ones                       |
| 2 ★         | $\leq 25$ vulnerabilities, $\leq 2$ of high severity, no critical ones |
| 3 ★         | $> 25$ vulnerabilities, $\leq 5$ of high severity, no critical ones    |
| 4 ★         | $\leq 10$ high vulnerabilities or $\leq 2$ critical ones               |
| 5 ★         | $> 10$ high vulnerabilities or $> 2$ critical ones                     |

Table 3.1: Project vulnerability rating done by SmartCheck.

A zero-star rating is obtained if Slither finds no security issue (informational and optimization issues are not included). A One-star rating is awarded if Slither finds less than 10 vulnerabilities, none of which are of high or critical severity. For a project to get a three-star rating, the project is required to have no critical vulnerability, but is



allowed to have up to five high severity vulnerabilities. If any critical vulnerability is found, the project will get a rating of four or five stars regardless of how many issues are found. Naturally, a project that earns a certain star rating must surpass the previous ratings thresholds (for example, to get a two star rating, a project must first exceed 10 vulnerabilities of any kind, which is the condition to get one star).

The idea behind this rating is to raise awareness to the developer towards the security threats and induce a sense of urgency towards solving the high and critical vulnerabilities. For this reason, any critical vulnerability found will grant a minimum of four stars, while projects with no high severity vulnerabilities will only get one star. Furthermore, while one could argue that the rating should factor in the project's complexity and allow more vulnerabilities for larger projects, we should also consider that high-scale projects may also get targeted more often by attackers while the financial stakes are also higher. As such, the state of a project's security should not be influenced by its size.

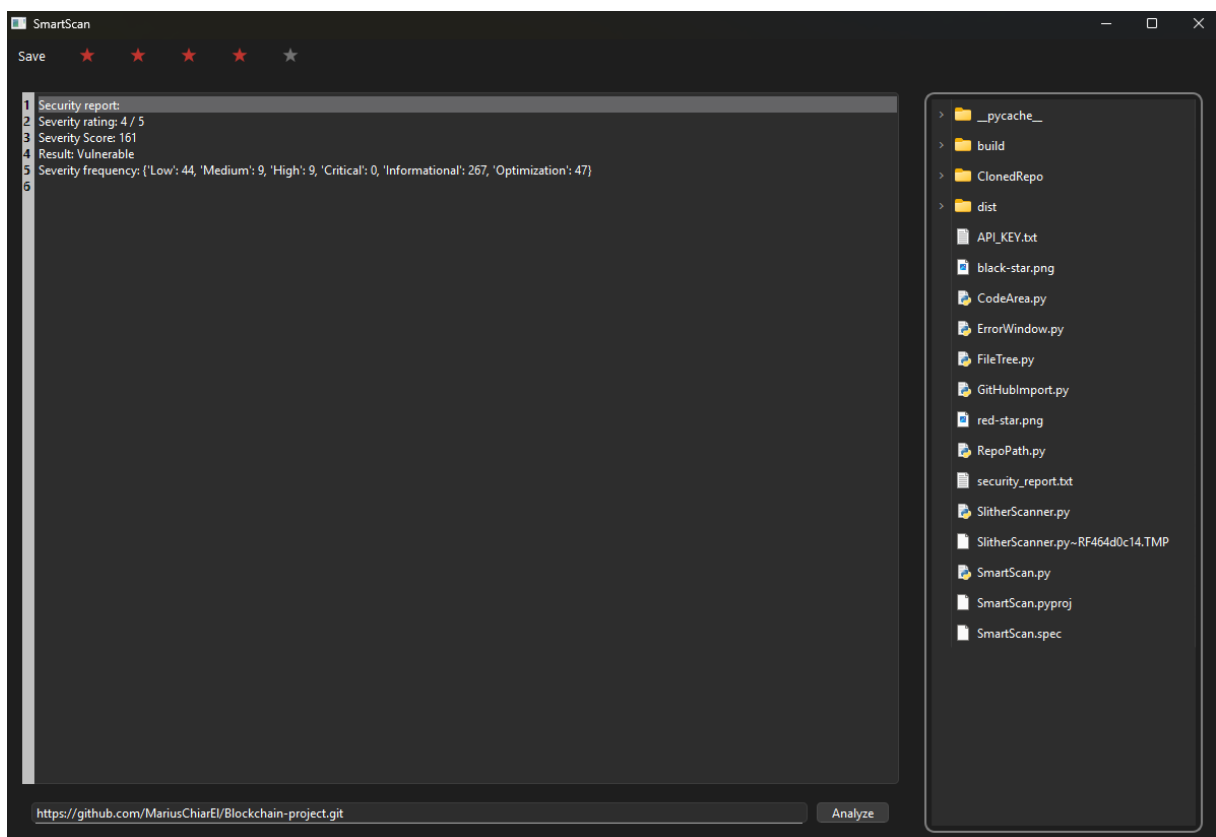


Figure 3.4: SmartScan's User Interface in practice: The Code Area features the final security report of a smart contract.

As shown in the screenshot above, our sample smart contract project obtained a rating of four stars. Despite having no critical security issue, the high severity vulner-

abilities are the root cause of these results. In this project, Slither had found 44 low severity errors, nine of medium severity and another nine of high severity. Additionally, the tool found 47 opportunities to optimize the smart contract and 267 informational errors, which are often not indicating any direct problems for the end user.

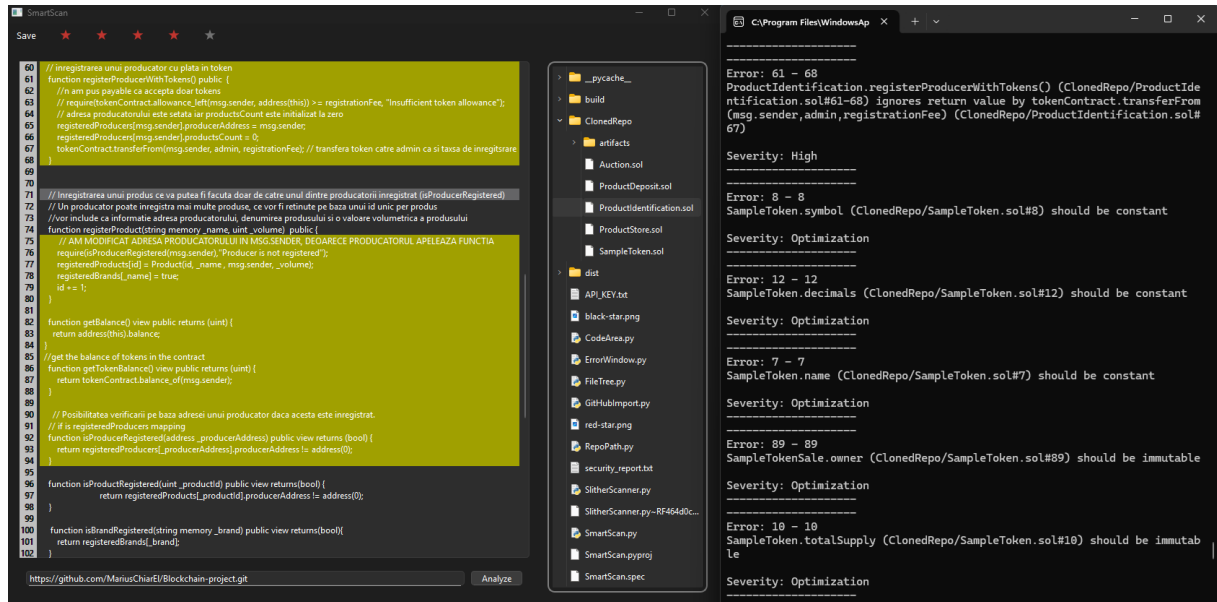


Figure 3.5: SmartScan’s User Interface in practice: The Code Area features the code of a Solidity file with its affected lines highlighted. The console shows the description of each error.

The figure above shows SmartScan in action, highlighting the errors, as reported in the console. For instance, we can see the `registerProducerWithTokens()` method, containing an error at line 67, where `transferFrom()` [26] was called. Slither detected that this function had been called without checking its result, as it returns a boolean that tells whether the transfer succeeded. In this case, the registration proceeds while the user does not have the funds to do so. This vulnerability has a high severity because any user can obtain the producer title without paying the smart contract as intended. This kind of vulnerability has the highest impact on the smart contract owner because they are the one to not get the funds they should have.

| Project         | Overall Security Rating | Errors (by severity) |        |      |          |       |
|-----------------|-------------------------|----------------------|--------|------|----------|-------|
|                 |                         | Low                  | Medium | High | Critical | Total |
| Project #1 [20] | 3/5 ★                   | 16                   | 10     | 3    | 0        | 29    |
| Project #2 [16] | 4/5 ★                   | 44                   | 9      | 9    | 0        | 62    |
| Project #3 [17] | 5/5 ★                   | 31                   | 0      | 11   | 0        | 21    |

Table 3.2: The security reports of three similar smart contracts.

This table shows the results SmartScan obtained by analyzing three similar smart contracts. The first project has the best results, having only three high severity vulnerabilities and a three star rating. The second one shows the highest amount of errors, caused primarily by the 44 low severity errors, but did not surpass the five star threshold of 10 high severity errors, so it earned four out of five stars. At the same time, while it has the least total errors, the third project has the most high severity errors, which grant it the five star rating.

| Project                | Time   |
|------------------------|--------|
| Sample Project #2 [16] | 6.56 s |
| Sample Project #3 [17] | 8.65 s |
| Sample Project #1 [20] | 9.16 s |

Table 3.3: Time needed for cloning and analyzing the three projects.

On the subject of SmartScan's speed, it will mostly be determined by Slither's high performance in terms of analysis and REST API's speed of cloning a project, which may be limited by the user's internet speed, especially when approaching complex smart contracts. Considering this application's aim to help students and beginner developers, we will stick to measuring the cloning and analysis speed of the same three projects from the previous table. For these measurements we used the internal stopwatch and took the average of three runs for each project to avoid anomalies. We can conclude that, for small projects, SmartScan can provide a reasonably quick analysis because all of these projects took less than 10 seconds to analyze across all the runs. The execution time of this project is especially important for a teacher who needs to evaluate their students' projects or assessments and needs to run the same process on tens of repositories.

Another important topic regarding SmartScan is its ability to guide a beginner

user through the analysis and audit process. The Code Area of the application is used to host messages that direct the user's attention to each step of the routine and provides explanations as to how to access and interpret the scanned code. This feature helps the user have a streamlined experience, paired with the relatively low run time for the analysis itself. Additionally, SmartScan uses a simple and familiar design, with the Code Area as its central component, which shares its space with the File Tree. The user may resize the two elements as needed or hide the File Tree completely to better focus on the code. Another user-friendly feature of SmartScan is showing various messages in case of failure (for instance, if the user enters an invalid repository link).

SmartScan achieves its goals to ease the audit process and guide its users to easily track the detected errors and optimizations by automizing most of the work, compiling an overall security report and highlighting the affected sections of the code while also naturally integrating into the development and testing process. Its qualities make it suitable in an academic setting, used by the students to see various coding patterns that lead to security vulnerabilities or inefficient usage of gas, or in a team of beginner developers who wish to provide the most secure solutions for their users. However, there are also multiple ways to improve the application and the user's experience. Firstly, a very useful feature would be generating a separate file that organizes the errors and helps the developer track the errors even more easily and to which they can add other errors, found through manual analysis or with the help of other tools. Additionally, Smart Scan could integrate the functions to review the differences, commit and push the improved code back to the repository to make the process as seamless as possible. Another improvement to this application can take the form of a separate window that shows the last clicked error so they can easily see the errors one by one. Needless to say, SmartScan is still a prototype, but it's able to achieve its goals at a basic level and has a clear path forward in terms of improvements.

# Conclusions

This thesis gravitated around three main subjects: blockchain's infrastructure, Slither's design and performance analysis and SmartScan, the application made to ease the security analysis process. Now we have a better picture of what blockchain is and how it functions, including qualities and shortcomings alike, potential attacks that affect this kind of networks and the currently available solutions to prevent them. The second chapter was dedicated to Slither, one of the best tools for security analysis, with emphasis on its infrastructure, functionality and performance compared to similar solutions. Lastly, we have introduced SmartScan, the application that integrates Slither to automatically detect vulnerabilities in remote projects. The description of SmartScan includes its infrastructure, feature set, workflow and examples of security reports generated by it. Our hope is that our contribution will enable aspiring developers to create secure smart contracts and that the blockchain will be safer to use in the long term as a consequence.

In conclusion, the blockchain is a very promising technology, with essential applications in various domains, that evenly distributes the power and responsibility to all its users. However, its security concerns keep its popularity low among most people and more security measures must be taken in order to restore its image and make it usable in key areas, such as electronic voting, where it could significantly improve the voting process by easing it for both the citizens and the states. Nevertheless, there is a long road ahead of blockchain towards achieving its potential and it will need a lot of support from the scientific community for that to happen, much like any other new piece of technology.

# Bibliography

- [1] Roberto Baldoni et al. "A Survey of Symbolic Execution Techniques". In: *ACM Comput. Surv.* 51.3 (2018). DOI: <https://doi.org/10.48550/arXiv.1610.00502>.
- [2] Senan Behan. "Solidity smart contract testing with static analysis tools". In: *Blockchain Research and Applications* (2022). URL: <https://norma.ncirl.ie/5932/1/senanbehan.pdf>.
- [3] Lyubka Dencheva. "Comparative analysis of static application security testing (SAST) and dynamic application security testing (DAST) by using open-source web application penetration testing tools". In: *International Journal of Computer Applications* (2022). URL: <https://norma.ncirl.ie/5956/1/lyubkadencheva.pdf>.
- [4] Pär Emanuelsson and Ulf Nilsson. "A Comparative Study of Industrial Static Analysis Tools". In: *Electronic Notes in Theoretical Computer Science* 217 (2008). Proceedings of the 3rd International Workshop on Systems Software Verification (SSV 2008), pp. 5–21. ISSN: 1571-0661. DOI: <https://doi.org/10.1016/j.entcs.2008.06.039>. URL: <https://www.sciencedirect.com/science/article/pii/S1571066108003824>.
- [5] Noama Fatima Samreen and Manar H. Alalfi. "Reentrancy Vulnerability Identification in Ethereum Smart Contracts". In: (2020), pp. 22–29. DOI: 10.1109/IWBOSE50093.2020.9050260. URL: <https://arxiv.org/pdf/2105.02881>.
- [6] Josselin Feist, Gustavo Grieco, and Alex Groce. "Slither: A Static Analysis Framework for Smart Contracts". In: (2019), pp. 8–15. DOI: 10.1109/WETSEB.2019.00008. URL: <https://arxiv.org/pdf/1908.09878>.

- [7] Merit Kõlvart, Margus Poola, and Addi Rull. “Smart Contracts”. In: (2016). Ed. by Tanel Kerikmäe and Addi Rull, pp. 133–147. DOI: 10.1007/978-3-319-26896-5\_7. URL: [https://www.researchgate.net/profile/Tanel-Kerikmae/publication/314538429\\_The\\_Future\\_of\\_Law\\_and\\_Technologies/links/58c32c8445851538eb809e8c/The-Future-of-Law-and-Technologies.pdf#page=142](https://www.researchgate.net/profile/Tanel-Kerikmae/publication/314538429_The_Future_of_Law_and_Technologies/links/58c32c8445851538eb809e8c/The-Future-of-Law-and-Technologies.pdf#page=142).
- [8] Leslie Lamport. “The part-time parliament”. In: *ACM Trans. Comput. Syst.* 16.2 (May 1998), pp. 133–169. ISSN: 0734-2071. DOI: 10.1145/279227.279229. URL: <https://doi.org/10.1145/279227.279229>.
- [9] Bahareh Lashkari and Petr Musilek. “Evaluation of Smart Contract Vulnerability Analysis Tools: A Domain-Specific Perspective”. In: *Information* 14.10 (2023). ISSN: 2078-2489. DOI: 10.3390/info14100533. URL: <https://www.mdpi.com/2078-2489/14/10/533>.
- [10] Kaixuan Li et al. “Static Application Security Testing (SAST) Tools for Smart Contracts: How Far Are We?” In: *Proc. ACM Softw. Eng.* 1.FSE (July 2024). DOI: 10.1145/3660772. URL: <https://doi.org/10.1145/3660772>.
- [11] Mark Mo et al. “Manticore: A user-friendly symbolic execution framework for binaries and smart contracts”. In: *arXiv preprint arXiv:1907.03890* (2019). DOI: <https://doi.org/10.48550/arXiv.1907.03890>.
- [12] Peng Qian et al. “Demystifying random number in Ethereum smart contract: Taxonomy, vulnerability identification, and attack detection”. In: *IEEE Transactions on Information Forensics and Security* (2023). DOI: <https://doi.org/10.48550/arXiv.2304.12645>.
- [13] Muhammad Saad et al. “Exploring the attack surface of blockchain: A systematic overview”. In: *IEEE Communications Surveys & Tutorials* (2019). DOI: <https://doi.org/10.48550/arXiv.1904.03487>.
- [14] Alberto Sonnino et al. “Replay attacks and defenses against cross-shard consensus in sharded distributed ledgers”. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 2020. DOI: <https://doi.org/10.48550/arXiv.1901.11218>.

- [15] Karl Wüst and Arthur Gervais. *Do you Need a Blockchain?* Tech. rep. 2018, pp. 45–54. DOI: 10.1109/CVCBT.2018.00011. URL: <https://eprint.iacr.org/2017/375.pdf>.



## Useful Links

- [16] Alexandra-Elena T., Ana-Maria O. and Marius E. *Sample smart contract #2*. <https://github.com/MariusChiarEl/Blockchain-project>. Accessed: 2025-06-13. 2023.
- [17] Alexandru C., Cristian-Ştefan B. and Eduard T. *Sample smart contract #3*. <https://github.com/cristibctr/SmartSupplyChain>, <https://github.com/cristibctr/Tema2Blockchain>. Accessed: 2025-06-13. 2023.
- [18] Digiconomist. *Ethereum energy consumption index*. <https://digiconomist.net/ethereum-energy-consumption>. Accessed: 2025-05-22. 2023.
- [19] Marius E. *SmartScan*. <https://github.com/MariusChiarEl/SmartScan>. Accessed: 2025-06-14. 2025.
- [21] Josselin Feist, Gustavo Grieco, and Alex Groce. *Slither GitHub Repository*. <https://github.com/crytic/slither>. 2018.
- [22] GitHub. *GitHub REST API documentation*. <https://docs.github.com/en/rest?apiVersion=2022-11-28>. Accessed: 2025-06-08. 2025.
- [23] Quadriga Initiative. *Quadriga Initiative*. <https://quadrigainitiative.com/indexjay.php>. Accessed: 2025-05-24.
- [20] Eduard-Antonio P. and Iustin-Ştefănel B. *Sample smart contract #1*. <https://github.com/IustinBarbir/Blockchain>. Accessed: 2025-06-13. 2023.
- [24] SmartState. *Understanding overflow and underflow vulnerabilities in smart contracts*. <https://smartstate.tech/blog/understanding-overflow-and-underflow-vulnerabilities-in-smart-contracts.html>. Accessed: 2025-05-22. 2023.
- [25] The Qt Company. *PySide6 documentation*. <https://doc.qt.io/qtforpython-6/>. Accessed: 2025-06-11. 2025.

- [26] The Solidity Authors. *Solidity documentation*. <https://docs.soliditylang.org/en/v0.8.30/>. Accessed: 2025-06-12. 2025.